Багатопоточність у Free Pascal: що нового

Гілка **feature/concurrency** (FPC 3.3.1) · жовтень 2026 · для розробників, які використовують System.Threading

Огляд

Гілка виправляє помилки в `System.Threading` (пакет vcl-compat), додає підтримку перевірки гонок даних через valgrind і робить тести багатопоточності надійнішими. Публічний API не змінено: існуючий код компілюється без змін і починає працювати коректніше.

Що	Було	Стало
`TParallel.For`	Іноді виконувався в одному потоці (20,7 с замість 1,7 с); ітерації могли губитися або виконуватися двічі	Стабільно паралельний; кожна ітерація рівно один раз
`TTask.Wait`, `WaitForAny`	Читання стану без бар'єра / без блокування	Коректно і на x86, і на ARM64
`TThreadPool.Free`	Повертав керування, поки потоки пулу ще працювали; запис у звільнену пам'ять монітором	Дочікується фактичного завершення всіх потоків
valgrind / helgrind	Сотні хибних попереджень; падіння valgrind	Модуль `fpvalgrind`, анотації, скрипти; 0 попереджень у System.Threading
Тести	`Assert` з робочих потоків міг загубити невдачу	`ThreadCheck` / `ThreadCheckSame`

1. Паралельний цикл TParallel.For

Виправлено дві помилки. Перша: на щойно створеному («холодному») пулі вільні потоки не помічали роботу в локальній черзі кореневої задачі й спали до 40 с, тож увесь цикл виконувався одним потоком. Друга: потік забирав діапазон ітерацій з одним розміром кроку, а виконував з іншим, тому частина ітерацій губилася або виконувалася двічі, а `TLoopResult.Completed` при цьому був `True`.

docs/concurrency/examples/parallel_for.pp

```
program parallel_for;
// TParallel.For: кожна ітерація виконується рівно один раз,
// цикл справді паралельний навіть на «холодному» пулі.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, System.Threading;

const
  N = 1000000;
var
  Hits: array of Integer;
  I, Bad: Integer;
  Res: TParallel.TLoopResult;
begin
  SetLength(Hits, N);
  Res := TParallel.&For(0, N - 1,
    procedure(aIndex: Integer)
    begin
      AtomicIncrement(Hits[aIndex]);   // різні потоки пишуть у різні елементи
    end);
  Bad := 0;
  for I := 0 to N - 1 do
    if Hits[I] <> 1 then
      Inc(Bad);
  WriteLn('Completed: ', Res.Completed, ', iterations not run exactly once: ', Bad);
end.
```

Вивід програми:

```
Completed: TRUE, iterations not run exactly once: 0
```

2. Задачі: TTask, IFuture, WaitForAll/WaitForAny

`Wait` перевіряє завершення без блокування (швидкий шлях); тепер це читання має бар'єр пам'яті, тож результат задачі видно одразу, і на процесорах зі слабким порядком пам'яті (ARM64) також. `WaitForAny` читає індекс завершеної задачі під тим самим блокуванням, під яким його записують.

docs/concurrency/examples/tasks.pp

```
program tasks;
// Задачі: TTask.Run, IFuture<T>, WaitForAll / WaitForAny.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, System.Threading;

var
  Answer: specialize IFuture<Integer>;
  Jobs: array[0..2] of ITask;
  First: Integer;
begin
  // Результат обчислюється в потоці пулу; Value чекає на нього.
  Answer := TTask.specialize Future<Integer>(
    function: Integer
    begin
      Sleep(50);
      Result := 6 * 7;
    end);
  WriteLn('Future value: ', Answer.Value);

  Jobs[0] := TTask.Run(procedure begin Sleep(300); end);
  Jobs[1] := TTask.Run(procedure begin Sleep(20);  end);
  Jobs[2] := TTask.Run(procedure begin Sleep(150); end);

  First := TTask.WaitForAny(Jobs);          // індекс першої завершеної
  WriteLn('First finished: task #', First);
  WriteLn('All finished: ', TTask.WaitForAll(Jobs, 2000));
end.
```

Вивід програми:

```
Future value: 42
First finished: task #1
All finished: TRUE
```

3. Власний пул потоків і його знищення

Потоки пулу тепер належать пулу (`FreeOnTerminate := False`). Завершуючись, потік переходить у список завершених, а пул дочікується його (`WaitFor`) і звільняє — у потоці-моніторі та обов'язково в `Destroy`. Тому після `Pool.Free` жоден потік пулу вже не виконується, а пул за замовчуванням не конфліктує з завершенням програми.

docs/concurrency/examples/pool_lifetime.pp

```
program pool_lifetime;
// Власний пул: Free повертає керування лише тоді, коли всі потоки пулу
// справді завершились (раніше вони могли ще працювати).
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, System.Threading;

// Кількість потоків процесу (лише Linux).
function LiveThreads: Integer;
var
  Info: TSearchRec;
begin
  Result := 0;
  if FindFirst('/proc/self/task/*', faDirectory, Info) = 0 then
  try
    repeat
      if (Info.Name <> '.') and (Info.Name <> '..') then
        Inc(Result);
    until FindNext(Info) <> 0;
  finally
    FindClose(Info);
  end;
end;

var
  Pool: TThreadPool;
  Done, I: Integer;
begin
  WriteLn('Threads before: ', LiveThreads);
  Pool := TThreadPool.Create;
  try
    Pool.SetMaxWorkerThreads(4);
    Done := 0;
    for I := 1 to 20 do
      Pool.QueueWorkItem(
        procedure
        begin
          Sleep(10);
          AtomicIncrement(Done);
        end);
    while Done < 20 do
      Sleep(1);
    WriteLn('Threads while pool is alive: ', LiveThreads);
  finally
    Pool.Free;            // чекає (WaitFor) і звільняє всі свої потоки
  end;
  WriteLn('Threads after Free: ', LiveThreads);
end.
```

Вивід програми:

```
Threads before: 1
Threads while pool is alive: 6
Threads after Free: 1
```

Зверніть увагу. Обробник `OnThreadTerminate` виконується в потоці, що завершується. `Destroy` чекає й на нього, тож повільний обробник уповільнює `Free`. Не викликайте з нього `TThread.Synchronize`, якщо пул знищується з головного потоку.

4. Перевірка гонок даних: модуль fpvalgrind

helgrind і drd не бачать синхронізації без `pthread_mutex`: спін-блокувань, атомарних операцій, кешу об'єктів. Модуль `fpvalgrind` (пакет fcl-base) передає valgrind підказки. Код анотацій є лише в збірці з `-dFPC_VALGRIND`; у звичайній збірці його немає. Запити реалізовано для x86_64 (System V); на інших платформах це порожні процедури.

Процедура	Призначення
`VgHappensBefore(p)`	Потік «публікує» все, що записав, з міткою p (перед звільненням блокування / записом прапорця)
`VgHappensAfter(p)`	Потік бачить усе, що опубліковано з міткою p (після захоплення блокування / читання прапорця)
`VgCleanMemory(p, n)`	Пам'ять «нова»: для об'єктів, перевикористаних поза менеджером пам'яті (кеші)
`VgBenignRace(p, n)`	Не перевіряти поле, яке свідомо читається без блокування (прапорці)
`VgRunningOnValgrind`	True, якщо програма працює під valgrind

docs/concurrency/examples/valgrind_annotate.pp

```
program valgrind_annotate;
// Анотації для helgrind/drd: власна передача даних через прапорець.
// Звичайна збірка:     fpc valgrind_annotate.pp
// Збірка з анотаціями: fpc -dFPC_VALGRIND valgrind_annotate.pp
{$mode objfpc}{$H+}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes{$IFDEF FPC_VALGRIND}, fpvalgrind{$ENDIF};

var
  Payload: Integer;     // дані, які передаємо
  Ready: LongInt;       // прапорець «дані готові»

type
  TProducer = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TProducer.Execute;
begin
  Payload := 42;
  {$IFDEF FPC_VALGRIND}
  VgHappensBefore(@Ready);          // усе записане вище «опубліковано»
  {$ENDIF}
  InterlockedExchange(Ready, 1);    // атомарний запис з бар'єром
end;

var
  P: TProducer;
begin
  {$IFDEF FPC_VALGRIND}
  WriteLn('Running on valgrind: ', VgRunningOnValgrind);
  {$ENDIF}
  P := TProducer.Create(False);
  while InterlockedCompareExchange(Ready, 1, 1) <> 1 do
    Sleep(1);
  ReadBarrier;
  {$IFDEF FPC_VALGRIND}
  VgHappensAfter(@Ready);           // бачимо все, що опублікував виробник
  {$ENDIF}
  WriteLn('Payload: ', Payload);
  P.WaitFor;
  P.Free;
end.
```

Вивід програми:

```
Running on valgrind: TRUE
Payload: 42
```

Результат helgrind для цього прикладу:

Збірка	Звіт helgrind
без анотацій	хибна гонка на Payload (рядок 42 ↔ рядок 22) + 1 відома особливість RTL у TThread.WaitFor
-dFPC_VALGRIND	лише відома особливість RTL у TThread.WaitFor

Анотації вже додано в TSpinLock (syncobjs) і в System.Threading. Для своїх програм: позначайте анотаціями лише ту синхронізацію, яку helgrind не бачить сам.

5. Тести: ThreadCheck замість Assert у потоках

Assert* у робочому потоці кидає виняток у цьому потоці, а не в тесті, тож невдача може загубитися; лічильники fpcunit не потокобезпечні. У TThreadedTestCase (тести System.Threading) є ThreadCheck і ThreadCheckSame: невдача записується, а TearDown після завершення всіх потоків валить тест з переліком невдач.

Фрагмент тесту

```
procedure TMyTest.DoWork(Sender: TObject);   // виконується в потоці пулу
begin
  // AssertSame('Sender', Self, Sender);     // НІ: виняток у чужому потоці
  ThreadCheckSame('Sender', Self, Sender);   // ТАК: невдачу покаже TearDown
  ThreadCheck(Counter > 0, 'Counter must be positive');
end;
```

Так виглядає невдача:

```
TestExecuteWork  Failed: [TEARDOWN] Checks failed in worker threads:
[thread 127149134571200] Counter must be positive
```

Тестові потоки (TNotifyThread, TTaskThread) тепер не FreeOnTerminate: TearDown дочікується їх і звільняє.

6. Інструменти (tests/concurrency/)

Скрипт	Що робить
fpcenv.sh	Налаштування для компіляції свіжим компілятором з дерева джерел (окремий файл налаштувань на кожен запуск)
build-testcompat.sh [тека]	Збирає тести vcl-compat з поточних джерел
stress.sh [N] [таймаут]	Запускає кожен набір тестів System.Threading N разів окремим процесом; виявляє рідкісні падіння й зависання
build-testcompat-valgrind.sh	Збірка з -dFPC_VALGRIND і обходом падіння valgrind на символах налагодження FPC
fpc-helgrind.supp	Винятки для навмисної поведінки RTL
hgsummary.py лог	Групує звіт helgrind за місцями конфліктів

Типовий цикл перевірки

```
tests/concurrency/build-testcompat.sh
tests/concurrency/out/testcompat --all --format=plain
tests/concurrency/stress.sh 50 60

tests/concurrency/build-testcompat-valgrind.sh
valgrind --tool=helgrind --suppressions=tests/concurrency/fpc-helgrind.supp \
  --log-file=hg.log tests/concurrency/out-valgrind/testcompat --suite=TTestTask
tests/concurrency/hgsummary.py hg.log
```

7. Зміни в RTL (Linux / Unix)

Файл	Зміна
`rtl/unix/tthread.inc`	Новий потік більше не перезаписує без синхронізації службовий прапорець `FSuspendedInternal` (його вже встановлено до старту потоку)
`rtl/unix/cthreads.pp`	Список ідентифікаторів потоків `ThreadTidList` читається лише під `ThreadTidLock`

Ці зміни стосуються кожної програми на FPC під Linux. Повний набір тестів компілятора (10 442 тести) дає той самий список падінь, що й до змін.

8. Перевірено

Перевірка	Результат
Тести vcl-compat	702 тести; 2 відомі падіння, не пов'язані з потоками
Стрес-прогін	9 наборів × 50 повторів: 450/450; після останніх змін у тестах — 20 повторів: 180/180. Без падінь і зависань
Тести компілятора	той самий список падінь, що й до змін
helgrind	0 попереджень у System.Threading; решта — відомі категорії RTL

9. Відомі обмеження

• helgrind не розуміє атомарного лічильника посилань `TInterfacedObject` (модуль System) і показує хибні конфлікти під час звільнення інтерфейсних об'єктів. На роботу програм не впливає.

• `TThread.WaitFor` опитує прапорець завершення без блокування; після нього `pthread_join` дає повну синхронізацію. Доброякісно.

• Анотації valgrind реалізовано лише для x86_64.

Докладніше: `docs/concurrency/valgrind.md`, `docs/concurrency/baseline.md`, `CHANGELOG-concurrency.md`.